ELTE IK - Computer Science BSc

Final Exam Topics

21. Databases - optimization and concurrency control

Note: This document was translated from Hungarian to English by AI. Please verify the information against the original Hungarian source before relying on it.

Databases – optimization and concurrency control

The task and parts of database management systems. Index structures, execution of queries, optimization strategies. Processing of transactions, logging and recovery, concurrency control.

1 The task and parts of database management systems

By a database management system we mean a computer program that realizes the safe storage, fast queryability, and modifiability of a large mass of data, typically for several users at once.

Database management activities are usually divided into two groups: data manipulation (querying), and definition (forming, modifying data structures). The languages serving the manipulation of data are collectively called Data Manipulation Language (DML), while the languages having definition tools are usually called Data Definition Language (DDL).

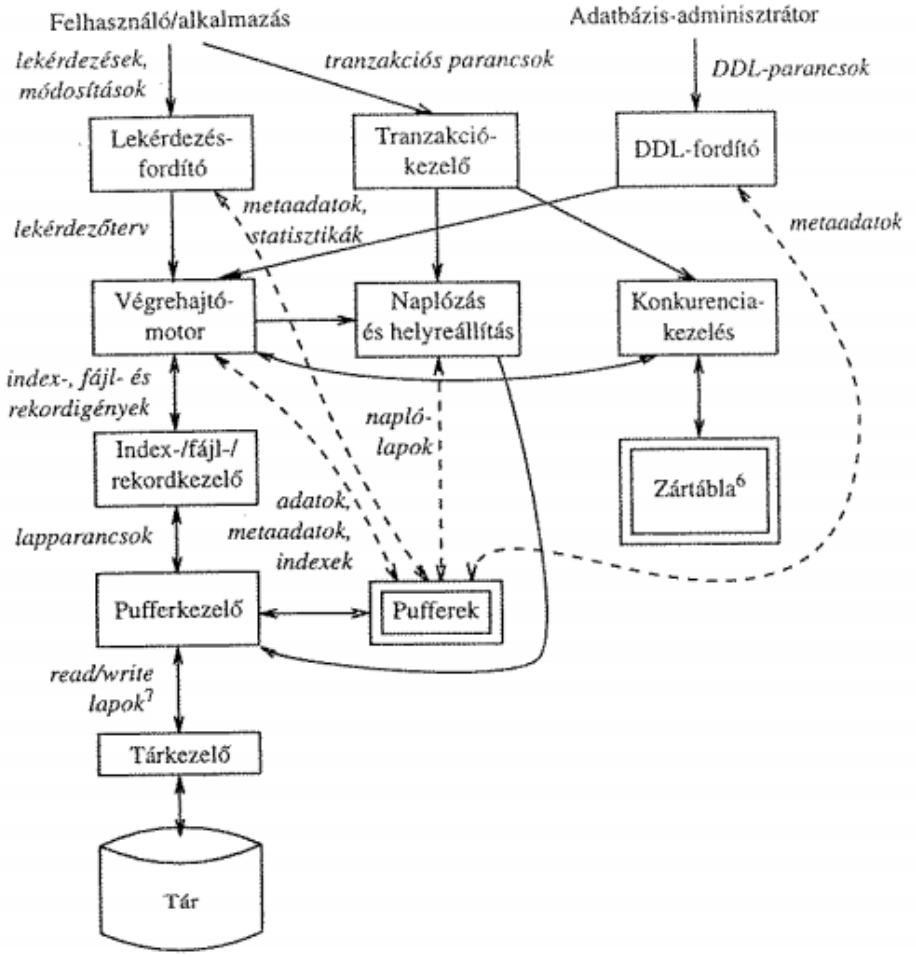

Figure 1: The components of the database management system.

1.1 Brief characterization of the individual components

Query compiler: The query compiler analyzes and optimizes the query, based on which it produces the query execution plan (query plan).

Execution engine: The execution engine receives the query plan from the query compiler, then passes to the resource manager a sequence of requests relating to smaller data pieces (typically records, rows of a relation).

Resource manager: The resource manager knows the data files containing the relations, the format and size of the records of the files, as well as the index files. The data requests the resource manager translates into pages, which it passes to the buffer manager.

Buffer manager: Its task is to read the appropriate part of the data from the secondary storage (disk, etc.) into the buffers of the central memory. The buffer manager exchanges information with the storage manager in order to obtain the data from the disk.

Storage manager: It reads and writes data on the secondary storage. It can happen that it also uses the OS's commands, but often it addresses its commands directly to the disk manager.

Transaction manager: We organize the queries and other activities into transactions. Transactions are units that have to be executed atomically and in an isolatable way, and the execution has to be durable, and the execution of the transaction must not produce an invalid database state (that is, consistent). These requirements are usually summarized with the ACID acronym. The transaction manager executes the transactions and takes care

of the logging and recovery, as well as the concurrency control.

2 Index structures

Indexes are auxiliary structures that speed up searching. An index can be made on several fields too. Not only the main file but also the index has to be maintained, which means extra cost. If the search field does not coincide with any index field then it means heap organization. The structure of index records is (a,p), where "a" is a value in the indexed column, "p" is a block pointer, pointing to the block in which the record with A=a value is stored. The index is always sorted by the index values.

2.1 Primary index

In the case of a primary index the main file is also sorted (by the index field), so because of this only one primary index can be given. It is enough to make an index record for the smallest record of every block of the main file, so their number is: $T(I) = B$ (sparse index). Much more index records fit into a block than main-file records: $bf(I) >> bf$, that is, the index file is a much smaller sorted file than the main file: $B(I) = \frac{B}{bf(I)} << B = \frac{T}{bf}$.

When searching, since not every value appears in the index file, we can only search for a covering value, the largest index value that is smaller than or equal to the searched value. The search for the covering value is done, because of the orderedness of the index, with binary search: $\log_2(B(I))$. The block corresponding to the block pointer in the covering index record still has to be read in. So the cost is $1 + \log_2(B(I)) << \log_2(B)$ (sorted case). At modification, the insertion has to be made into the sorted file. If the first record changes in the block, then the index file also has to be inserted into, which is likewise sorted. The solution is that we leave empty places in the blocks of the main file and the index file too. With this the storage size can double, but the insertion means at most one main record and one index record write-back.

2.2 Secondary index

In the case of a secondary index the main file is unsorted (the index file is always sorted). Several secondary indexes can be given. An index record has to be made for every record of the main file, so the number of index records is: $T(I) = T$ (dense index). Much more index records fit into a block than main-file records: $bf(I) >> bf$, that is, the index file is a much smaller file than the main file: $B(I) = \frac{T}{bf(I)} << B = \frac{T}{bf}$. The search in the index is done, because of the orderedness of the index, with binary search: $\log_2(B(I))$. The block corresponding to the block pointer in the found index record still has to be read in. So the cost is $1 + \log_2(B(I)) << \log_2(B)$ (sorted case). The search time is worse than for the primary index, because there are more index records. The main file is heap-organized. The insertion has to be made into the sorted file. If the first record changes in the block, then the index file also has to be inserted into, which is likewise sorted. Solution: we leave empty places in the blocks of the main file and the index file too. With this the storage size can double, but the insertion means at most one main record and one index record write-back.

2.3 Bitmap index

Bitmap indexes are usually assigned to given values of columns, in the following way:

- If in the column the value of the i-th row coincides with the given value, then the i-th member of the bitmap index is a 1.
- If in the column the value of the i-th row, however, does not coincide with the given value, then the i-th member of the bitmap index is a 0.

Thus for a query one only has to appropriately AND or OR the bitmap indexes, and in the number sequence thus obtained find where there is a 1. The binary values are usually compressed with run-length encoding for the sake of more efficient storage.

2.4 Multi-level indexes

The index file (1st index level) is also a file, moreover a sorted one, so this too can be indexed, with a primary index. The main file can be sorted or unsorted (the index file is always sorted). The t-th level index: we also index the index levels, up to t levels in total. On the t-th level ($I(t)$) we search for the covering index record with binary search. We follow the pointer, on every level, and finally in the main file: $\log_2(B(I(t))) + t$ block reads. If the topmost level consists of 1 block, then it means $t + 1$ block reads. The blocking factor of every level coincides, because the index records are of equal length.

On the t-th level 1 block: $1 = \frac{B}{bf(I)^t}$. That is, $t = \log_{bf(I)}(B) < \log_2(B)$, so it is better than sorted file organization. The $\log_{bf(I)}(B) < \log_2(B(I))$ also generally holds, so it is also faster than single-level indexes.

2.5 B-tree index

Logically the index is a sorted list. Physically the sorted order is ensured by pointers arranged into a table. The tree-structured indexes can be represented with B-trees. B-trees solve the unbalancing problem of binary trees, since we fill them "from below". Several key values can belong to one node of a B-tree. The pointers point to other nodes, and thus to all key values on the given node.

Since B-trees are balanced (every branch is of equal length, that is, ends on the same level), they eliminate the variable access times that can be observed in binary trees. Although the key values and the addresses associated with them are still found on every level of the tree, and the result of this is: unequal access paths, and unequal access time, as well as a complex tree-search algorithm for the logically serial reading of the data file. This can be eliminated if we do not allow the storage of data-file addresses above the leaf level. Consequently: every access takes a path of the same length, the result of which is equal access time, and a logically serial reading of the data file can be solved by reaching the leaf level. There is no need for a complex tree-search algorithm.

The B^+-tree is a B-tree that is at least 50% filled. Besides the structure, we also include the maintenance algorithms ensuring the filledness.

The B^*-tree is a B-tree that is at least 66% filled.

2.6 Hash index

The records are located partitioned into buckets, over which the function defined on the h hash field distributes them. It is characterized by efficient equality search, insertion, and deletion, but it does not support interval search.

We sort the records into block chains, and put the records into the first empty place of the last block of the block chain in the order of arrival. The number of block chains can be given in advance (static hashing, in which case we denote the number by K), or can change based on the stored data (dynamic hashing). The sorting is done based on the values of the index field. The value of a hash function $h(x) \in \{1, .., K\}$ tells which bucket the record belongs to, if x was the value of the index field in the record. The hash function is generally based on division with remainder (for example $mod(K)$). A hash function is good if roughly equally long block chains arise, that is, it sorts the records evenly. Then the block chain consists of $\frac{B}{K}$ blocks.

The cost of searching:

- If the index field and search field differ, then it means heap organization.
- If the index field and search field coincide, then it is only enough to go through the bucket with sequence number $h(a)$, which corresponds to a heap of $\frac{B}{K}$ blocks, that is, $\frac{B}{K}$ in the worst case. The search thus speeds up by a factor of K.

The storage size is B if every block is roughly full. For large K, however, there can be many block chains that will consist of one block, and even in the block there will be only 1 record. Then the search time is: 1 block read, but instead of B we store the data in T blocks. At modification, the bucket with $\frac{B}{K}$-block heap organization has to be modified.

3 Execution and optimization of queries

The query processor is the group of components of a relational database manager that translates the user's queries, as well as data-modifying statements, into database operations, which it also executes.

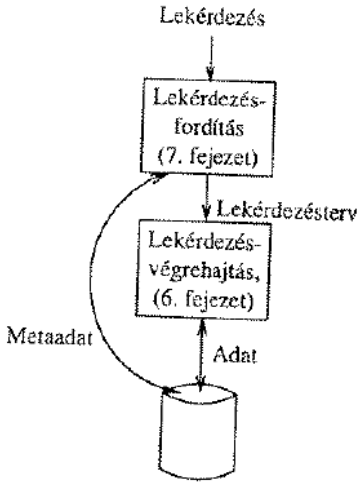

Figure 2: The steps of query processing.

The execution of a query is essentially the totality of the algorithms manipulating the database, but before execution the queries need to be compiled.

The steps of query compilation:

1. Parsing: we build up a parse tree that characterizes the query and its structure
2. Query rewriting: From the parse tree we make an initial query plan, which we transform into an equivalent plan whose execution time is expected to be smaller. The logical query plan is produced, which contains relational algebra expressions.
3. We transform the logical plan into a physical plan by choosing an algorithm for the operators of the logical plan, and determining the execution order of the operators. The physical plan also contains such details as e.g. whether sorting is needed, how we access the data.

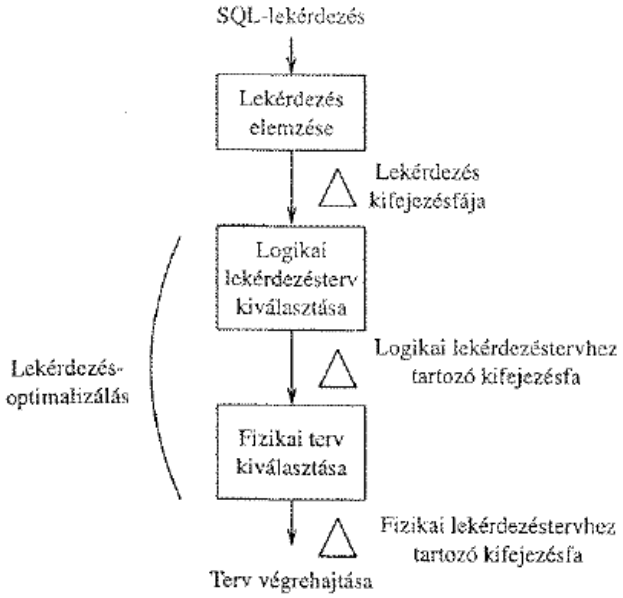

Figure 3: The steps of query compilation.

3.1 Algebraic optimization

We want to compute the relational algebra expressions as fast as possible. The cost of the computation is proportional to the sum of the storage sizes of the relations corresponding to the subexpressions of the relational algebra expression. The method is that we apply equivalent transformations based on operation properties, so that expectedly smaller-size relations arise. The procedure is heuristic, so it does not compute with the real size of the argument relations. The result is not unambiguous, since the order of the transformations is non-deterministic, so executing the transformations in a different order we can get a different end result, but each is generally of better cost than the one we started from.

The optimizing algorithm is based on the following heuristic principles:

- Let us select as early as possible, so that the subexpressions are expectedly smaller relations.
- From the selections after a multiplication let us try to form natural joins, because the join can be computed more efficiently than a selection from a product.
- Let us combine the consecutive unary operations (selections and projections), and from these let us form, if possible, one selection, or projection, or a projection after a selection. Thus the number of operations decreases, and generally the selection results in a smaller relation than the projection.
- Let us look for common subexpressions, which it is then enough to compute only once during the evaluation of the expression.

3.2 Realization of relational algebra operations

3.2.1 Selection

The possible realizations of selection (σ):

- Linear search: Let us read in every page and look for the matches (in the case of an equality test). The average cost is the number of pages if the field is not a key, and half the number of pages if the field is a key.
- Binary (logarithmic) search: Can be used only in the case of a sorted field.
- Use of a primary index.
- Use of a secondary index.

Compound selection: It can happen that there are several conditions, which are in an and/or connection with each other. Its realization:

- In the case of a conjunctive selection ($\sigma_{\theta_1 \wedge \ldots \wedge \theta_n}$):
  - Let us choose the smallest-cost σ_{θ_i}, and perform that (see above), then filter the result for the other conditions. The cost will be the cost of the simple selection for the chosen σ_{θ_i}.
  - If we have an index on the field of every θ_i, then let us search in the indexes and return the rowids of the appropriate rows. Finally let us take their intersection. The cost is the total cost of searching in the indexes + the reading in of the records.
- In the case of a disjunctive selection ($\sigma_{\theta_1 \vee \ldots \vee \theta_n}$):
  - Linear search.
  - If we have an index on the field of every θ_i, then let us search in the indexes and return the rowids of the appropriate rows. Finally let us take their union. The cost is the total cost of searching in the indexes + the reading in of the records.

3.2.2 Projection and set operations

In projection and set operations the duplicates have to be filtered out.

Realization of projection (π):

1. Initial scan: we drop the unnecessary fields.
2. Deletion of duplicates: for this we sort the result by all fields, so the duplicates become adjacent. These have to be dropped.

The cost will be the total cost of the initial scan, the sorting, and the deletion of duplicates.

3.2.3 Joins

Joins can be performed in several ways:

- **Nested loop join**: Can be used for relations of any size, it is not necessary for one relation to fit in the memory. It has two kinds, the row-based and the block-based.
  - **Row-based nested loop join**: Let R and S be the two relations to be joined; of these let S denote the smaller-size one, this will be the inner relation, and R the outer. The algorithm is the following: let us go through all rows of R (the outer relation) and at each row of R let us go through all rows of the inner relation, S. If the 2 currently examined rows can be joined, then let us write them into the result. In the good case S fits in the memory, in which case S has to be read in only once, then we can keep it in the memory throughout. In this case the pages of both relations have to be read in once. In the worst case only one page each fits in the memory from both relations. Then at each row of R, S has to be read through.
  - **Block-based nested loop join**: The difference compared to the previous is that we do not go through the relations row by row. Let M be the number of pages fitting in the memory. Then in each iteration we read in M-1 pages of R, then organize these into some search structure where the key is the common attributes of R and S. After this let us go page by page through S, and from the rows of S in the memory let us look for those that can be joined with some row read in from R, then write these into the result relation.
- **Merge sort join (merge join)**: The relations are sorted by the join fields. We merge the sorted relations: pointers to the first record in both relations. We read in a record group from S where the value of the join attribute coincides. We read in records from R and process them. The sorted relations have to be read through only once, so the cost of the join is: cost of sorting + the number of pages of the 2 relations.
- **Hash join**: Let us use the join attribute as a hash key, based on this let us partition the rows of R and S with hashing. If we want n buckets from each table, then the result is: $R_0, R_1, ..R_{n-1}, S_0, S_1, .., S_{n-1}$ buckets. After this we join the matched bucket pairs with a block-based nested loop join, using a hash-function-based index.

Joining several tables: Joins are commutative and associative, so from the point of view of the result it does not matter in what order we join them. The order, however, can affect efficiency, since in case of a bad choice the intermediate results will be of large size.

Finding the best join tree for a set of n relations:

- To find the best join tree for a set S of n relations, let us take all possible plans that look like this: $S_1 \bowtie (S - S_1)$, where S_1 is any nonempty subset of S.
- Let us recursively compute the costs of the joins of the subsets of S, in order to determine the cost of every single plan. Let us choose the cheapest.

- When the plan of any subset has been computed, instead of recomputing it let us store it and reuse it when it is needed again.

4 Transaction management

Consistent database: Various constraints can be given on databases. A database is in a consistent state if it satisfies all such constraints. A consistent database is a database that is in a consistent state.

The consistency can be violated in the following cases:

- Transaction error: incorrectly written, badly scheduled, abandoned transactions.
- Database-management error: some component of the database manager does not perform its task, or performs it badly.
- Hardware error: a piece of data is lost, or its value changes.
- Error arising from data sharing.

Transaction: A sequence of consistency-preserving database operations. After this we always assume that if at the start of transaction T the database is in a consistent state, then if T runs alone, the database will be in a consistent state at the end of the run (in the meantime an inconsistent state can arise).
Condition of correctness:

- If one or more transactions stop (due to abort or error), even then we get a consistent database.
- Every single transaction sees a consistent database at start.

We usually expect the following properties from transactions (ACID):

- Atomicity (A): the "all or nothing" type execution of the transaction (either we execute it completely, or we do not execute it at all).
- Consistency (C): the condition that the transaction preserve the consistency of the database, that is, that after the execution of the transaction the consistency constraints prescribed in the database also hold.
- Isolation (I): the fact that every transaction has to seemingly run as if during this time we were not executing any other transaction.
- Durability (D): the condition that once a transaction has finished, the effect exerted by the transaction on the database can never be lost.

We always take consistency for granted. The other three properties, however, the database management system has to ensure, but from this we occasionally disregard. We assume that the database consists of data units, elements. A database element is a kind of functional unit of the data stored in the physical database, whose value can be accessed (read out) or modified (written out) with transactions. A database element is, e.g., the relation, relation row, page.

4.1 The basic activities of transactions

The interaction of the transaction and the database has 3 important locations:

- The area of the disk blocks containing the elements of the database.
- The virtual or real memory area used by the buffer manager.
- The memory area of the transaction.

For a transaction to be able to read in a database element, it first has to be brought into memory buffer(s), if it is not there yet. After this the transaction can read in the content of the buffer(s) into its own memory area. The writing out of the new value of a database element happens in reverse: first the transaction forms the new value in its own memory area, then this value is copied into the appropriate buffer(s).

The writing of the content of the buffers to disk is decided by the buffer manager. It either immediately writes the changes to disk, or not.

During the study of the logging algorithms and other transaction-management algorithms, various notations will be needed, with which we can describe the data movements between the various areas. We use the following basic operations:

- INPUT(X): Copying the disk block containing the database element X into the buffer.
- READ(X,t): Copying the database element X into the local variable t of the transaction. If the block containing X is not yet in the memory, then INPUT(X) is also included.
- WRITE(X,t): The content of the local variable t is copied into the memory-buffer content of the database element X. If the block containing X is not yet in the buffer, then INPUT(X) is also executed first.
- OUTPUT(X): Writing the block containing the database element X to disk.

In what follows we assume that a database element is not larger than one block.

4.2 Logging and recovery

The data has to be protected from system errors, so the recoverability of the data is needed. For this the primary technique is logging, which records the history of the modifications performed in the database with some safe method.

The log is nothing other than a sequence of log records, which contain information about what a transaction did. In case of a system error, with the help of the log it can be reconstructed what the transaction did up to the occurrence of the error.

4.2.1 Log records

We have to regard the log, as a file, as being open exclusively for appending. When executing a transaction, it is the task of the log manager to record every important event in the log.

The log records used by all logging methods:

- $\langle START\ T \rangle$: This record indicates the start of the execution of transaction T.
- $\langle COMMIT\ T \rangle$: Transaction T finished properly, it no longer wants to perform further modifications.
- $\langle ABORT\ T \rangle$: Transaction T aborted, it could not finish successfully. The changes made by it do not have to be copied to disk, or if they were copied to disk, then they have to be restored.

4.2.2 Undo logging

The essence of undo logging is that if it is not certain that the operations of a transaction finished properly and every change was written to disk, then the effect of the transaction has to be undone, that is, the database has to be restored to a state as if the transaction had never even started.

For undo logging one more kind of log record is needed, the modification record, which is a triple $\langle T, X, v \rangle$, and means that transaction T modified the database element X, and the value of X before the modification was v.

The rules of undo logging:

- If transaction T modifies the database element X, then the $\langle T, X, v\rangle$ log record has to be written to disk before the system writes the new value to disk.
- If the transaction finished without error, then the COMMIT record can be written to disk only after every modification performed by the transaction was written to disk.

Recovery using undo logging

Suppose a system error occurred. Then it can happen that a transaction was not executed atomically, that is, certain of its modifications were already written to disk, but others not yet. Then the database can get into an inconsistent state. Therefore in case of a system error the restoration of the consistency of the database has to be taken care of. In the case of undo logging this means the undoing of the modifications performed by the unfinished transactions.

Restoration without a checkpoint: The simplest method. Then we see the whole log. The first task is the division of the transactions into successfully finished and unfinished transactions. A transaction T finished successfully if there is a $\langle COMMIT\ T\rangle$ record in the log. Then T in itself could not have left the database in an inconsistent state. If we find a $\langle START\ T\rangle$ record in the log, but no $\langle COMMIT\ T\rangle$ record, then we can assume that T performed a modification in the database that was not yet written to disk. Then T is not a complete transaction, its effect has to be undone.
The algorithm is the following: The recovery manager starts to examine the log records from the last one, proceeding backwards, while it records those T transactions for which it found a $\langle COMMIT\ T\rangle$ or $\langle ABORT\ T\rangle$ record. Proceeding backwards, when it sees a $\langle T, X, v\rangle$ log record:

1. If it already encountered a COMMIT record for T, then it does nothing.
2. Otherwise T is incomplete or aborted. Then the recovery manager changes the value of the database element X to v.

After performing the above changes, for every incomplete transaction T it writes $\langle ABORT\ T\rangle$ to the end of the log and triggers the writing of the log to disk. After this the normal use of the database can continue.

4.2.3 Checkpointing

Recovery would in principle require the examination of the whole log. If we use undo logging, then if for a transaction T there is a COMMIT record in the log, then the records relating to transaction T are not needed for recovery, but it is not necessarily true that we can delete the records relating to transaction T before the COMMIT. The simplest solution is to make checkpoints from time to time.

Making a simple checkpoint

1. Stopping requests for starting new transactions.
2. Waiting for the still-active transactions to finish and for the COMMIT/ABORT record to be written into the log.
3. Writing the log to disk.
4. Forming the $\langle CKPT\rangle$ log record, writing it into the log, then writing the log to disk.
5. Restarting the service of transaction-start requests.

The transactions executed before the checkpoint have finished, their modifications got to disk. Therefore it is enough to analyze the part of the log after the last checkpoint at recovery.

Creating a checkpoint during the operation of the system

The problem with simple checkpointing is that it does not allow the starting of new transactions until the active transactions have finished. This, however, can still take a lot of time, and to the user the system seems stopped, since they cannot start a new transaction. This we cannot allow. A more complex method, however, makes checkpointing possible without having to suspend the starting of new transactions.

The steps of this method:

1. Making a $\langle START\ CKPT(T_1, T_2, ...T_k)\rangle$ record and writing the log to disk. $T_1, ..T_k$ are the currently active, unfinished transactions.
2. The finishing of the $T_1, ..T_k$ transactions has to be waited for. Meanwhile new transactions can be started.
3. If every one of the $T_1, ..T_k$ transactions still active at the start of the checkpointing has finished, then making an $\langle END\ CKPT\rangle$ log record and writing it to disk.

Recovery: Analyzing backwards we find the unfinished transactions, and restore the content of the database elements modified by these transactions to the old value. Two cases can occur, we encounter either an $\langle END\ CKPT\rangle$ or a $\langle START\ CKPT(T_1, T_2, ...T_k)\rangle$ log record first.

- If we encounter an $\langle END\ CKPT\rangle$ record first, then the records relating to all unfinished transactions can be found up to the nearest $\langle START\ CKPT(T_1, T_2, ...T_k)\rangle$ record. With those earlier than this we do not have to deal.
- If we encounter a $\langle START\ CKPT(T_1, T_2, ...T_k)\rangle$ record first, then the error happened during checkpointing. Then, of the $T_1, ..T_k$ transactions, we have to go back to the START record of the earliest started one, which, however, is certainly found after the START CKPT record preceding this.

As a general rule, if we write END CKPT to disk, then the records preceding the START CKPT record preceding it are not needed from the point of view of recovery.

4.2.4 Redo logging

Redo vs. undo logging:

- Redo logging, in contrast to undo logging, at recovery disregards the unfinished transactions and finishes the changes performed by the normally finished ones.
- In the case of undo logging we require the writing of the modifications to disk before the writing of COMMIT into the log. In contrast, in the case of redo logging we write the modifications performed by the transaction to disk only if the COMMIT record was written into the log and got to disk.
- In undo logging the old value of the modified elements is needed at recovery, and in redo the new.

The rules of redo logging: Here let the $\langle T, X, v\rangle$ record mean that transaction T changed the value of the database element X to v. The order in which the data and log records have to get to disk is determined by the following so-called "write earlier" logging rule: Before we modify any element X of the database on disk, it is necessary that the $\langle T, X, v\rangle$ and $\langle COMMIT\ T\rangle$ log records get to disk.

Recovery using redo logging:

If for a transaction T there is no $\langle COMMIT\ T\rangle$ record in the log, then we know that T's modifications did not get to disk, so we do not have to deal with these. If, however, T finished, that is, there is a $\langle COMMIT\ T\rangle$ record, then either its modifications got to disk, or not. Therefore T's modifications have to be repeated. Fortunately the log records contain the new values.

The steps of recovery in case of a catastrophe:

1. Determine those transactions for which there is a COMMIT record in the log.
2. Analyze the log starting from the beginning. If we find a $\langle T, X, v\rangle$ record, then if T is a finished transaction, then the value v has to be written into X. If T is unfinished, we do nothing.
3. If we reached the end of the log, then for every unfinished transaction T we write a $\langle ABORT\ T\rangle$ log record into the log and write the log to disk.

Redo logging with checkpointing

For redo logging the checkpointing during operation consists of the following steps:

1. Making a $\langle START\ CKPT(T_1, T_2, ...T_k)\rangle$ log record and writing it to disk, where $T_1, ...T_k$ are the active transactions.
2. Writing to disk all those database elements that were written into the buffer by transactions that finished (COMMIT) before $\langle START\ CKPT(T_1, T_2, ...T_k)\rangle$, but whose buffers did not yet get to disk.
3. Making an $\langle END\ CKPT\rangle$ log record and writing it to disk.

Restoration in redo logging supplemented with a checkpoint: Two cases can occur: the last log record related to the checkpoint is either START CKPT or END CKPT.

- Suppose the last checkpoint record in the log is END CKPT. Then the modifications of the transactions that finished before $\langle START\ CKPT(T_1, T_2, ...T_k)\rangle$ certainly already got to disk, with these we do not have to deal, but with the T_i and the transactions started after the START CKPT record we have to deal. Then we have to do such a restoration as in plain redo logging, with the difference that only the T_i and the transactions started after the START CKPT have to be taken into account. In the search we have to go back to the earliest $\langle START\ T_i\rangle$ record, where T_i is one of the $T_1, ...T_k$.
- Suppose the last checkpoint record in the log is $\langle START\ CKPT(T_1, T_2, ...T_k)\rangle$. We cannot be sure that the modifications of the transactions that finished before this were written to disk, so we have to go back to the $\langle START\ CKPT(S_1, S_2, ...T_m)\rangle$ before the END CKPT preceding this. We have to restore the modifications of those transactions that are among the S_i, or started after $\langle START\ CKPT(S_1, S_2, ...T_m)\rangle$ and finished.

4.2.5 Undo/redo logging

In undo/redo logging the modification-indicating log records are of the form $\langle T, X, v, w\rangle$, which means that transaction T changed the value of the database element X from v to w.

In the case of undo/redo logging the following prescription has to be observed: Before we modify the value of any element X of the database on disk, the $\langle T, X, v, w\rangle$ log record has to get to disk.

In particular the $\langle COMMIT\ T\rangle$ record can both precede and follow the writing of the modifications to disk.

Recovery in undo/redo logging

The basic principles:

1. Starting from the earliest, let us restore the effect of every finished transaction.
2. Starting from the last, let us undo the effect of the unfinished transactions.

Undo/redo logging with checkpointing:

1. Let us write a $\langle START\ CKPT(T_1, T_2, ...T_k)\rangle$ record into the log, where $T_1, ..T_k$ are the active transactions, then write the log to disk.

2. Let us write to disk those buffers that contain modified database elements (dirty buffers). In contrast to redo logging, here we write every dirty buffer to disk, not only those of the finished ones.

3. Let us write an $\langle END\ CKPT \rangle$ log record into the log and write it to disk.

4.3 Concurrency control

The interaction between transactions can cause the database to become inconsistent, even when the transactions individually preserve the consistency and no system error occurred either. Therefore we have to somehow regulate in what order the individual steps of the different transactions follow each other. This is done by the scheduler, and the process itself is called concurrency control.

Our basic assumption (correctness principle) is that if we execute every single transaction separately, then they preserve the consistent database state. In practice, however, the transactions generally run concurrently, so the correctness principle cannot be applied directly. Thus we have to consider such schedules that ensure that they produce the same result as if we had executed the transactions one after another, one by one.

4.3.1 Schedules

Schedule: A schedule is the time-ordered sequence of the essential operations performed by one or more transactions. For schedules we only deal with the write and read operations.

Serial schedule: A schedule is serial if for any transactions T and T', if T has an operation that precedes some operation of T', then every operation of T precedes every operation of T'. The serial schedule is given by the listing of the transactions, e.g. (T_1, T_2).

Serializability: A schedule is serializable if it has the same effect on the database state as some serial schedule, independently of the initial state of the database.

Notations:

- $w_i(x)$ means that transaction T_i writes the database element x.
- $r_i(x)$ means that transaction T_i reads the database element x.

Conflict: There is a conflict if there is a consecutive operation pair in the schedule whose order, if we swap it, then the behavior of at least one transaction changes. Suppose T_i and T_j are different transactions. Then there is no conflict if the pair is:

- $r_i(X)$ and $r_j(Y)$, even if $X = Y$. That is, read operations performed by 2 different transactions are never in conflict with each other, even if they relate to the same database element.
- $r_i(X)$ and $w_j(Y)$, if $X \neq Y$
- $w_i(X)$ and $r_j(Y)$, if $X \neq Y$
- $w_i(X)$ and $w_j(Y)$, if $X \neq Y$

There is a conflict if:

- Any two operations of the same transaction are in conflict, since these cannot be swapped.
- The same database element is accessed by two different transactions, and of these at least one is a write operation.

So operations of different transactions are not in conflict with each other, that is, they are swappable, unless they

1. relate to the same database element, and
2. at least one of the operations is a write

Conflict-equivalent schedules: Two schedules are conflict-equivalent if they can be carried into each other by the non-conflicting swapping of adjacent operations.

Conflict-serializable schedules: A schedule is conflict-serializable if it is conflict-equivalent to some serial schedule. Conflict-serializability is a sufficient but not necessary condition of serializability. In commercial systems conflict-serializability is checked.

Precedence graph: Given a schedule S of the transactions T_1 and T_2, possibly of further transactions too. T_1 precedes T_2 in S if there is an operation A_1 in T_1 and an operation A_2 in T_2 for which:

1. A_1 precedes A_2 in S,
2. A_1 and A_2 relate to the same database element, and
3. at least one is a write operation

These are exactly the conditions when A_1 and A_2 are in conflict, are not swappable. These precedences we can illustrate with a precedence graph. The nodes of the precedence graph are transactions in S. If we denote the transactions by T_i, let i be the node belonging to T_i in the graph. A directed edge goes from node i to node j if T_i precedes T_j.

Relationship of the precedence graph and conflict-serializability: A schedule S is conflict-serializable if and only if its precedence graph is acyclic. Then any topological sorting of the vertices of the precedence graph gives a conflict-equivalent serial schedule.

4.3.2 Locks

Conflict-serializability can also be achieved with the use of locks. If the scheduler uses locks, then the transactions have to request and release locks in addition to the reading and writing of the database elements. The use of locks has to be correct in two senses:

- Consistency of transactions: The operations and the locks are connected to each other according to the following expectations:
  1. A transaction can read or write an element only if it has already locked it earlier, and has not yet released the lock.
  2. If a transaction locks an element, then it has to release it later.
- Legality of schedules: The meaning of the locks should correspond to the intended expectation: two transactions cannot lock the same element, only in such a way that one of them has already released the lock earlier.

Notations:

- $l_i(x)$: Transaction T_i requests a lock on the database element x.
- $u_i(x)$: Transaction T_i releases the lock of the database element x.

Locking scheduler: The task of the locking scheduler is to allow the requests if and only if it results in a legal schedule. In this the lock table helps, which for every database element gives that transaction, provided there is one, which is currently locking the given element.

Two-phase locking: We speak of two-phase locking (2PL) if in every transaction every locking operation precedes every release operation. Those transactions that satisfy 2PL we call 2PL transactions. A legal schedule of consistent, 2PL transactions is conflict-serializable.

Deadlock: We speak of a deadlock if the scheduler forces a transaction to wait forever for a lock that another transaction holds locked. A typical example of the formation of a deadlock is if 2 transactions want to lock elements locked by each other. Two-phase locking cannot prevent the formation of deadlocks.

		Kért	zár
		S	*X*
Érvényes zár	*S*	Igen	Nem
ebben a módban	*X*	Nem	Nem

Figure 4: Example of a deadlock.

Locking systems with different lock modes

Shared and exclusive locks: Any database element can be locked either once exclusively (for writing), or several times shared (for reading). An exclusively locked element cannot be locked shared, and a shared-locked element cannot be locked exclusively. If we want to write X, then we have to have an exclusive lock on X; if we want to read, then a shared one is also enough, but we can also read with an exclusive lock.

Compatibility matrix: The compatibility matrix has a row and a column for every single lock mode. The rows correspond to the locks already valid on element X by another transaction, and the columns correspond to the lock modes requested on X. The rule of use: a C-mode lock can be allowed if and only if for every row R of the table for which another transaction has already locked X in mode R, "Yes" appears in the C column.

		Kért	zár
		S	*X*
Érvényes zár	*S*	Igen	Nem
ebben a módban	*X*	Nem	Nem

Figure 5: Compatibility matrix of shared and exclusive locks.